OSCAL Artifact Integrity

Verifiable integrity for JSON artifacts and connected OSCAL packages in compliance-trestle

August 2026 | GSoC

From assumed evidence to verifiable evidence

Before

Auditors must trust:

- Files are identified mainly by names and locations.
- Auditors must assume they received the versions that were reviewed.
- Related documents can change independently.

After

Auditors can verify:

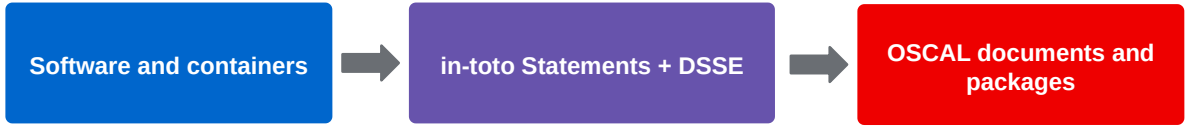
- The current files match the content approved by the signer.
- One proof can cover an exact set of related documents.
- Later content or package changes are detected.

Community benefit

Trestle turns document identity from an assumption into a verifiable claim using standardized signing and attestation formats.

OSCAL integrity follows proven supply-chain patterns

Trestle applies established software-signing approaches to compliance evidence.



Established practice

Organizations already sign software releases and container images to verify integrity and signer identity under an established trust policy.

Standard building blocks

in-toto Statements describe the approved artifacts, while DSSE provides the signed envelope.

Applied to OSCAL

Trestle combines these standards with canonical JSON and OSCAL dependency discovery.

Compliance evidence must prove what was actually reviewed

OSCAL decisions depend on connected documents, not one isolated file.

- An SSP depends on profiles, catalogs, and other OSCAL artifacts.
- The SSP can remain unchanged while one of those dependencies changes.
- Signing only the SSP proves only that document's canonical content.
- Teams need evidence for the exact artifact set reviewed together.

The risk

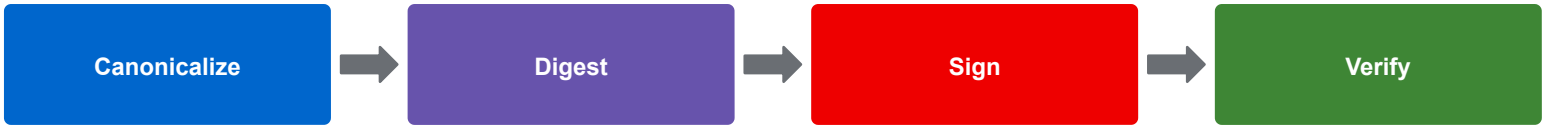
A valid SSP signature can coexist with a changed profile or catalog.

Evidence required

Verification must prove that the exact current artifact set and its canonical SHA-256 digests match the signed package claim.

Four steps create practical trust

The same integrity model works for one file and for a dependency package.



Stable artifact identity

RFC 8785 removes formatting differences so equivalent JSON produces the same bytes and digest.

Portable proof

A detached DSSE envelope containing an in-toto Statement records what was approved without modifying the original JSON.

Actionable failure

Verification rejects altered content, wrong keys, and unexpected claims; package mode checks every listed artifact.

The same JSON content should verify as the same artifact

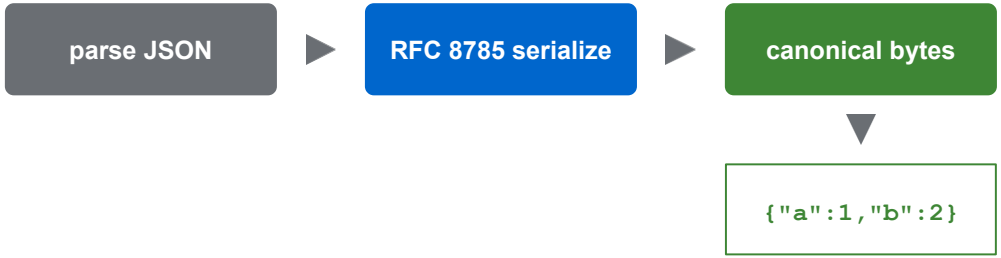
RFC 8785 removes editor formatting from the integrity decision.

Input A (Pretty Printed)

```
{
  "b": 2,
  "a": 1
}
```

Input B (Minified)

```
{"a":1,"b":2}
```



Reliable Integrity Decision

- Whitespace and key order no longer create false failures.
- The digest represents canonical JSON content, not one editor's serialization.
- Ambiguous or invalid JSON is rejected before approval.

Signing creates evidence without changing the artifact

The private-key holder signs a typed claim about the canonical JSON digest.

What the signer approves

- A DSSE signature covers the payload type and the exact in-toto Statement.
- The Statement names the artifact and its canonical SHA-256 digest.
- The original JSON remains unchanged.

trestle sign

```
trestle sign \
-f catalog.json \
--private-key private.pem \
-o catalog.json.dsse
```

The detached proof travels beside the original JSON

Teams can store, exchange, and verify evidence without rewriting OSCAL.

catalog.json.dsse

```
{
  "payloadType": "application/vnd.in-toto+json",
  "payload": "<base64-encoded in-toto Statement>",
  "signatures": [
    {
      "keyid": "<signing-key identifier>",
      "sig": "<base64-encoded signature>"
    }
  ]
}
```

Interoperable format

DSSE identifies an in-toto JSON Statement using formats established for software-supply-chain attestations.

Explicit claim

The Statement carries the artifact name, canonical digest, predicate type, and digest rules.

Independent verification

Consumers verify the proof with the expected public key. The private key never leaves the signer.

Key metadata

keyid is a hint inside the envelope. Trust comes from the public key supplied by the verifier.

Signed claims are explicit and machine-checkable

Artifact identity stays separate from the rules used to produce its digest.

Decoded DSSE payload: in-toto Statement

```
{
  "_type": "https://in-toto.io/Statement/v1",
  "subject": [{
    "name": "catalog.json",
    "digest": { "sha256": "..." }
  }],
  "predicateType": ".../oscal-signing/v1",
  "predicate": {
    "canonicalization": "RFC8785",
    "digestAlgorithm": "sha256",
    "digestSource": "canonical-json",
    "tool": "compliance-trestle"
  }
}
```

Subject

Names the artifact and records its canonical SHA-256 digest. It stays generic because sign supports any JSON file.

Predicate

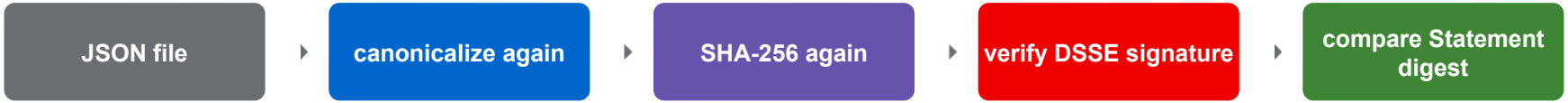
Records RFC 8785, SHA-256, canonical JSON as the digest source, and compliance-trestle as producer.

Verification value

Trestle rejects unexpected Statement types, predicate types, algorithms, subject counts, names, or digests.

Verification detects content change, not formatting noise

Equivalent JSON still passes; tampering or incompatible signed claims fail.



What teams can rely on:

Whitespace or key-order changes do not invalidate approval.

Changed JSON content changes the digest and fails.

Wrong keys or modified DSSE signatures fail.

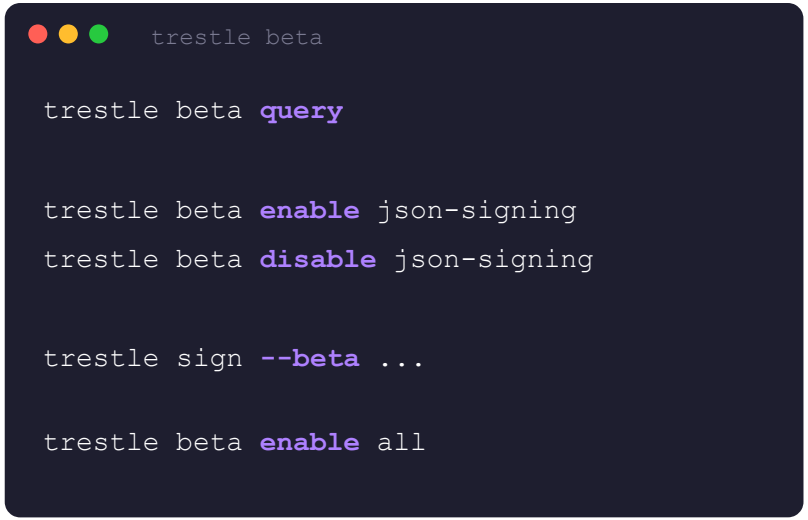
Unexpected Statement, predicate, or subject structures fail.

Verification does not modify the artifact.

```
trestle verify
trestle verify \
-f catalog.json \
--signature catalog.json.dsse \
--public-key public.pem
```

A reusable beta framework enables controlled experimentation

New capabilities can mature behind explicit opt-in without disrupting stable workflows.



Created for this project

The reusable beta framework adds feature discovery, enable/disable controls, and one-time --beta execution.

Used by integrity features

json-signing gates single-file signing and verification.

json-manifest-signing gates package generation, signing, and verification.

Reusable by Trestle

Future commands can use the same opt-in mechanism before their interfaces become stable.

Practical demo path

The same flow works locally, in release automation, or as a CI gate.

1. Generate keys

```
openssl genpkey -algorithm ed25519 -out private.pem
openssl pkey -in private.pem -pubout -out public.pem
```

2. Sign JSON

```
trestle sign -f catalog.json --private-key private.pem -o catalog.json.dsse
```

3. Verify JSON

```
trestle verify -f catalog.json --signature catalog.json.dsse --public-key public.pem
```

Real-world confidence

Exercised with a large NIST SP 800-53 catalog and negative cases for tampering, malformed JSON, wrong keys, and incompatible claims.

Operational key handling

Ed25519 and RSA PEM keys are supported; encrypted-key passwords come from an environment variable, not the command line.

Package verification detects dependency changes

A signed SSP is incomplete evidence if its profile or catalog can change independently.

Verification mechanics

- **Scope limitation:** A single-file signature protects only the SSP's canonical JSON content.
- **Dependency mapping:** generate-manifest follows the OSCAL dependency graph.
- **Strict binding:** sign-manifest binds the exact artifact set and canonical digests.
- **Drift detection:** verify-manifest fails when the manifest, signed claim, or any listed artifact no longer matches.

Product result

Teams exchange the artifacts, an inspectable package definition, and a detached proof that pins the exact compliance content reviewed together.

trestle verify-manifest

```
trestle verify-manifest \
--manifest manifest.json \
--signature package.dsse \
--public-key public.pem
```

One signed package answers key operational questions

Which artifacts were reviewed, which exact versions, and whether the current package still matches.

inspectable package

manifest.json

```
{
  "primaryArtifact": "ssp.json",
  "artifacts": [
    {"name": "ssp.json",
    "uri": "ssp.json",
    "mediaType": "application/oscal+json"},
    {"name": "profile.json",
    "uri": "profiles/profile.json",
    "mediaType": "application/oscal+json"}
  ]
}
```

What belongs

The manifest declares the primary artifact, package-relative identities, and exact file list.

What still matches

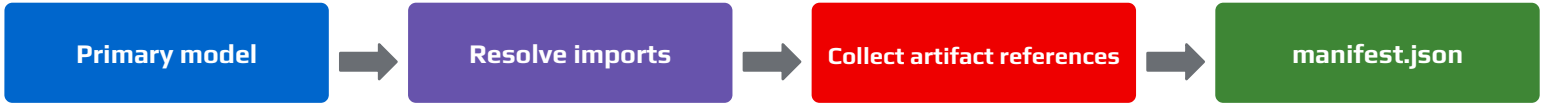
The Statement carries every canonical digest. Verification checks the declared list and each file's digest.

trestle sign-manifest

```
trestle sign-manifest \
--manifest manifest.json \
--private-key private.pem \
-o package.dsse
```

Trestle builds the package from OSCAL references

Start with one model, generate an inspectable manifest before anyone signs.




```
trestle generate-manifest -f ssp.json -o manifest.json
```

Semantic traversal

Loads generated OSCAL models and follows only dependency fields defined by each document type.

Handles graph complexity

Resolves paths from each source, deduplicates shared dependencies, rejects cycles, and preserves stable order.

Keeps approval explicit

Writes package-relative identities; teams inspect the JSON and use --include for content that cannot be inferred.

One workflow covers the top-level OSCAL JSON model family

Package generation can begin from authoring, assessment, or remediation artifacts.

Catalog + profile

Component + SSP

Assessment plan + results

POA&M + mapping collection

Model-aware discovery

Trestle validates each artifact and follows the relationships defined for that model, then stops at catalogs because this workflow follows no further dependencies from them.

Works with real repositories

Local, trestle://, HTTPS, SFTP, and UUID back-matter references resolve through one graph.

Tested beyond toy JSON

A real NIST SSP, profile, and catalog complete generate-sign-verify; model-specific tests cover every supported root.

Remote references become stable package evidence

Remote dependencies become stable, inspectable package artifacts.

Controlled acquisition

HTTPS and SFTP dependencies are downloaded through Trestle fetchers, canonicalized, validated as OSCAL, and saved inside the package.

Safer defaults

Direct loopback, link-local, metadata, redirects, and private-network targets are rejected by default; trusted private access requires explicit opt-in.

Bounded work

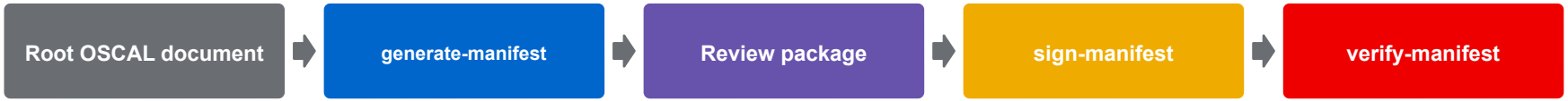
Limits graph depth, artifact count, each download, and total remote bytes.

Repeatable evidence

URI-derived names avoid collisions; traversal, symlinks, and silent replacement are rejected.

Teams review the package before they approve it

Discovery is automated, but signing remains an explicit decision.



What verification proves

- The DSSE signature verifies with the supplied public key.
- Manifest metadata and exact artifact list match.
- Every current artifact matches its signed canonical digest.

Current boundaries

- One signature covers the complete package claim, artifacts are not signed individually.
- Signer identity, trusted-key policy, and CI enforcement remain external decisions.

Adding signed evidence to the OSCAL Compass demo

The demo connects OSCAL authoring with automated assessment and compliance posture.

SSP release

A GitHub Actions workflow generates the manifest, signs and verifies the SSP package, and publishes it as a release asset.

Compliance-posture release

The workflow verifies the selected SSP release before packaging it with assessment evidence. Consumers verify both signature layers separately.

Proposed integration

- End-to-end demo
- SSP PR #9
- Compliance-posture PR #5

The GSoC outcome: portable integrity evidence from one file to a package

Trestle turns JSON and OSCAL dependency graphs into independently verifiable evidence.

Single artifact

Stable canonical digest + detached proof.

Connected package

Automatic discovery + exact artifact set + one package proof.

Controlled resolution

Local and bounded remote dependencies become inspectable evidence.

Ready now in beta

Sign and verify individual JSON artifacts, and generate, sign, and verify packages through beta-gated CLI commands.

Next decisions

Policy for per-document signatures, Sigstore-based identity, and broader CI enforcement across multi-party compliance workflows.